BEDROCK ARCHITECTURE

# Bedrock C ABI

C Language Binding

Version 0.1

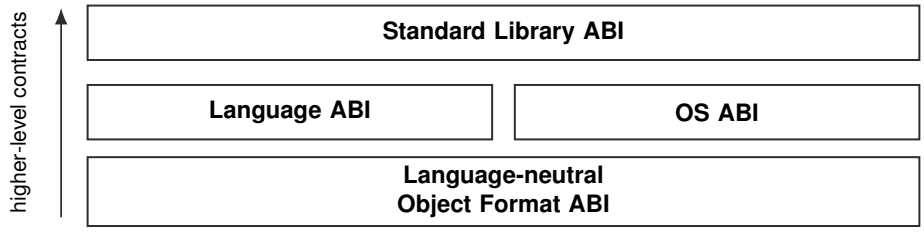
# Contents

# List of Tables

# 1 Scope and Profiles

The Bedrock C ABI defines the C data model, register convention, function call contract, aggregate passing rules, variadic call rules, return-value assignment, and compiler-runtime obligations for C-family toolchains. It builds on the Bedrock language-neutral ELF ABI for object files, relocation, loading, dynamic linking, TLS, and code models.

## 1.1 ABI Hierarchy




**Current document.** This document specifies one language ABI layer: the Bedrock C language binding. It relies on the language-neutral ELF ABI for object files, relocation, loading, dynamic linking, TLS, and code models, while leaving process-entry policy to an OS ABI.

## 1.2 ABI Profiles

**Table 1. Reference ABI Profiles**

| Profile | Meaning |
|---|---|
| `bedrock-c` | C-family language binding layered on the Bedrock language-neutral ELF ABI. |

# 2 Terminology

These terms are specific to the C binding. ELF and address-space terms keep the meanings defined by the language-neutral ABI and the architecture manual.

**object pointer** A 64-bit pre-segment address value that designates an object or one-past an object according to the C abstract machine.

**function pointer** A 64-bit pre-segment code address naming a near-call entry point. It is not a CS:PC pair, far descriptor, or runtime-private veneer descriptor.

**caller-saved register** A register whose value may be overwritten by a call unless the caller saves it before the call.

**callee-saved register**
A register whose incoming value must be preserved by the called function if the function uses it.

**sret buffer** Caller-allocated storage used to receive a return value that is too large or unsuitable for the normal return registers.

**addressable by-value argument**
A by-value aggregate argument that is passed through caller-owned stack storage because the callee may need an address for the object.

# 3 Data Model

**Model:** LP64

**Byte Order:** little endian

**Instruction Word:** 16 bits

**Stack Alignment:** 16 bytes

**Aggregate Maximum Alignment:** 16 bytes

## 3.1 Scalar Types

**Table 3. Reference C Data Model**

| Type | Bits | Align |
|---|---|---|
| `_Bool` | 8 | 1 |
| `char` | 8 | 1 |
| `signed char` | 8 | 1 |
| `unsigned char` | 8 | 1 |
| `short` | 16 | 2 |
| `int` | 32 | 4 |
| `long` | 64 | 8 |
| `long long` | 64 | 8 |
| `__int128` | 128 | 16 |
| `unsigned __int128` | 128 | 16 |
| `pointer` | 64 | 8 |
| `size_t` | 64 | 8 |
| `ptrdiff_t` | 64 | 8 |
| `wchar_t` | 32 | 4 |
| `char16_t` | 16 | 2 |
| `char32_t` | 32 | 4 |
| `float` | 32 | 4 |
| `double` | 64 | 8 |
| `long double` | 64 | 8 |

## 3.2 Layout Rules

- Integer and pointer objects are stored little-endian.
- Plain char is signed. signed char, unsigned char, and plain char remain distinct C types.
- The default enum representation is the 32-bit int representation with 4-byte alignment.
- Bool stores canonical values as 0 or 1 in bit 0; upper bits are ignored by consumers.
- int128 and unsigned int128 objects are stored little-endian; the low 64 bits live at the lower address.
- long double uses the same 8-byte IEEE-754 binary64 storage format as double.

- Pointer values are pre-segment virtual address values. Memory accesses apply segment pre-translation when the pointer is used as part of an effective address.
- C function pointers are ordinary 64-bit pre-segment code addresses naming near-call entry points. They are not CS:PC pairs or far-call descriptors.
- Object pointers and function pointers have the same size and alignment but remain distinct C pointer categories.
- Multiword immediate data and data directives store the least significant 16-bit word first.
- Struct fields are laid out in declaration order with natural alignment up to the aggregate maximum.
- Unions have the size and alignment of their most strictly aligned member, rounded up to that alignment.
- Arrays store elements contiguously with each element using the ABI size of its type.
- Aggregate object size is rounded up to the aggregate object's alignment.
- Boolean objects store the truth value in bit 0; upper bits are ignored by consumers.

# 4 Register Convention

**Table 5. Register Preservation and Assignment**

| Group | Rule | Registers / Meaning |
|---|---|---|
| integer registers | caller saved | D0, D1, D2, D3, D4, D5 |
| integer registers | callee saved | D6, D7 |
| integer registers | c scalar return | D0 |
| integer registers | c 128 bit scalar return | D1:D0 |
| integer registers | integer arguments | D0, D1, D2, D3, D4, D5 |
| address registers | caller saved | A0, A1, A2, A3, A4, A5 |
| address registers | callee saved | A6, A7 |
| address registers | pointer arguments | A0, A1, A2, A3, A4, A5 |
| address registers | c pointer return | A0 |
| address registers | conditional sret buffer argument | A5 |
| address registers | sret result return | A0 |
| floating point registers | caller saved | F0, F1, F2, F3, F4, F5, F6, F7 |
| floating point registers | callee saved | F8 |
| | | F9 |
| | | F10 |
| | | F11 |
| | | F12 |
| | | F13 |
| | | F14 |
| | | F15 |
| floating point registers | callee save mechanism | instruction pair: FPUSHM/FPOPM bitmap rule: bits 0..15 name F0-F15; C ABI prologues and epilogues normally set only bits for clobbered F8-F15 |
| floating point registers | floating point arguments | F0, F1, F2, F3, F4, F5, F6, F7 |
| floating point registers | c floating point return | F0 |
| stack register | name | SP |
| stack register | role | architectural stack pointer |
| stack register | preserved by callee | true |
| stack register | ordinary c use | stack pointer only; not available as a general C temporary |
| program counter | name | PC |
| program counter | role | architectural instruction pointer |
| program counter | ordinary c use | control-flow target only; not available as a general C temporary |
| flags | caller saved | FLAGS |
| flags | callee obligation | callees may clobber condition codes |
| status registers | STATUS | not part of the C abstract machine; ordinary C code may access it only through target-specific intrinsics or runtime code |
| status registers | FSTATUS | not part of the C abstract machine; floating-point environment support is a higher-level runtime contract |
| status registers | FFLAGS | not part of the C abstract machine; floating-point exception flag handling is a higher-level runtime contract |

## 4.1 Data Register Banking

The C ABI is DBANK-neutral. C function calls use bank zero as the ABI-visible D-register file; nonzero data-register banks are optional target resources for compiler-generated temporaries and specialized runtime code.

**Table 7. C ABI Data Register Banking Rules**

| Rule | Value |
|---|---|
| public boundary | DBANK must be 0 on C function entry and return |
| call boundary | A caller must issue an ordinary C call with DBANK = 0; a callee must return with DBANK = 0 |
| source language visibility | DBANK is not visible to the C abstract machine |
| bank zero | D0-D7 in bank 0 are the ABI-visible integer registers listed below |
| nonzero banks | caller-managed volatile scratch state when enabled by the target object attributes |
| preservation | no nonzero bank is callee-saved by the C ABI |
| live across call | C code must not keep a live value only in a nonzero DBANK across an ordinary C call |
| target feature gate | use of DBANK beyond zero requires the language-neutral ELF DBANK object attribute |

**Recommended compiler policy.**

- Do not use DBANK in the baseline C code-generation mode.
- Enable DBANK use only under an explicit target feature or optimization option.
- Prefer nonzero banks for leaf hot regions, spill-heavy loops, and REP or REPG lowering.
- Restore DBANK to zero before calls, returns, tail calls, unwinding edges, and externally visible labels.

# 5 Calling Convention

## 5.1 Stack and Calls

The stack grows downward. An architectural `CALL` pushes an 8-byte return address at `[SP]`, and callers align `SP` before the call so callee entry observes 16-byte alignment. The ABI reserves no red zone and no mandatory outgoing argument home area. A callee that needs stricter local alignment realigns its own frame and restores the incoming `SP` before return.

## 5.2 Argument Assignment

Arguments are assigned in source order. Scalar integer values use the D-register class, pointer and address values use the A-register class, and floating-point values use the F-register class. Arguments that exhaust their register class are placed in 16-byte stack slots.

**Table 9. C Argument Register Classes**

| Class | Registers |
|---|---|
| Integer scalar | `D0`, `D1`, `D2`, `D3`, `D4`, `D5` |
| Pointer/address | `A0`, `A1`, `A2`, `A3`, `A4`, `A5` |
| Floating-point | `F0`, `F1`, `F2`, `F3`, `F4`, `F5`, `F6`, `F7` |

**Table 11. C Stack Argument Rules**

| Rule | Value |
|---|---|
| Stack order | right to left |
| Slot size | 16 bytes |
| Slot alignment | 16 bytes |
| Narrow signed integer | sign-extend to 64 bits |
| Narrow unsigned integer and `_Bool` | zero-extend to 64 bits |
| `__int128` register pair | `D1:D0`, with low bits in `D0` and high bits in `D1` |
| `__int128` stack alignment | 16 bytes |
| Stack value placement | low address bytes |

## 5.3 C Return Values

**Table 13. C Return Register Assignment**

| Result Class | Register Rule |
|---|---|
| Integer scalar | `D0` |
| Pointer/address scalar | `A0` |
| Floating-point scalar | `F0` |
| `long double` scalar | `F0` |
| `__int128` scalar | `D1:D0` |
| Small aggregate | `D1:D0` |
| Large aggregate | sret pointer in `A5`, result pointer returned in `A0` |

## 5.4 Aggregate Passing

By-value aggregate arguments are addressable call copies. The caller creates the copy, passes its address, and keeps the copy valid until the call returns. Mixed integer/floating-point aggregates, packed aggregates, bitfield-containing aggregates, and under-aligned aggregates use the same addressable-copy argument rule.

**Table 15. Aggregate Argument and Return Rules**

| Rule | Value |
|---|---|
| By-value argument copy alignment | 16 bytes |
| Copy address registers | `A0`, `A1`, `A2`, `A3`, `A4`, `A5` |
| Aggregate register arguments | none |
| Small aggregate return | 16 bytes or smaller in `D1:D0` |
| Small aggregate byte layout | low address bytes D0 then D1 |
| Large aggregate return buffer alignment | 16 bytes |
| sret hidden pointer register | `A5` |
| sret result return register | `A0` |

## 5.5 Variadic Calls

Fixed named arguments use the normal assignment rules. Unnamed variadic arguments and old-style unprototyped call arguments are materialized in uniform stack slots, so `va_list` traversal does not depend on the fixed argument register assignment.

**Table 17. Variadic Argument Rules**

| Rule | Value |
|---|---|
| Variable arguments | stack only |
| Register save area | none |
| Variadic slot size | 16 bytes |
| Variadic slot alignment | 16 bytes |
| `long double` variable argument | 8-byte binary64 value in one 16-byte slot |
| Aggregate variable argument | copy address in slot |
| `va_arg` alignment | 16 bytes |
| `va_list` | next variadic stack slot pointer |

# 6 Freestanding C Binding

Freestanding C uses the Bedrock C data and call ABI together with the language-neutral ELF ABI. It is sufficient for compiler output, static objects, and runtime-provided helper calls, but it does not define process startup or a hosted C library environment.

## 6.1 Execution Boundary

The ELF entry field remains an object-loading field. The initial stack, `_start` contract, argument vectors, environment vectors, auxiliary vectors, and process-entry state are OS ABI responsibilities. Zero-fill for BSS storage is provided by ELF program loading for the file-size to memory-size gap in loadable segments.

## 6.2 Compiler Runtime Helpers

The baseline backend may expand simple `__int128` operations into 64-bit instruction sequences. Multiplication, division, remainder, and integer/floating-point conversions may instead lower to compiler-runtime helper calls. The ABI fixes the helper call convention, not a hosted runtime library surface: helper calls use the normal Bedrock C ABI, and `__int128` helper returns use `D1:D0`.

**Table 19. `__int128` Runtime Helper Names**

| Operation | Helper |
|---|---|
| Multiply | `__multi3` |
| Signed divide | `__divti3` |
| Unsigned divide | `__udivti3` |
| Signed remainder | `__modti3` |
| Unsigned remainder | `__umodti3` |
| Shift left | `__ashlti3` |
| Arithmetic shift right | `__ashrti3` |
| Logical shift right | `__lshrti3` |

Floating-point helper calls may be used when the selected target lacks a hardware operation. Those calls also use the normal Bedrock C ABI.

## 6.3 TLS and OS ABI Boundary

TLS requires an OS ABI, loader profile, or runtime that initializes the language-neutral TLS model. An unknown freestanding OS profile may leave TLS unsupported. TLSDESC and TLS relocation semantics are owned by the language-neutral ELF ABI and resolved by the linker, loader, or OS ABI.

## 6.4 C Linkage

C function pointers are 64-bit pre-segment near-call code addresses. Code and data pointers have the same size and alignment but remain distinct C pointer categories. Common symbols and externally visible objects are aligned to at least the ABI alignment of their declared type,

subject to the aggregate maximum unless stricter alignment is explicitly requested. Direct C calls use `R_BEDROCK_CALL_TARGET`; PLT-bound external calls use the PLT relocation selected by the active code model.

Instruction relaxation is assembler and linker policy under the active code model. Hosted C library entry points, process environment, filesystem behavior, and syscall bindings are outside the freestanding C ABI.

# 7 C Memory Model

The C ABI defines the memory guarantees that C-family toolchains may assume when lowering normal memory accesses, C atomic operations, volatile accesses, and compiler-runtime helper calls. It does not define device ordering policy or process-wide memory mapping policy.

## 7.1 Normal Memory

Naturally aligned 1, 2, 4, and 8 byte normal-memory loads and stores are tear-free. Unaligned accesses have no tear-free guarantee, and 16-byte or wider ordinary accesses are not atomic as single C ABI operations.

## 7.2 C Atomics

Native lock-free C atomics are available only for naturally aligned integer and pointer objects with sizes 1, 2, 4, 8 bytes. Unaligned atomic objects, 16-byte atomic objects, `_Atomic __int128`, and wider objects lower to libatomic or compiler-runtime helpers. `__atomic_always_lock_free` returns true only for naturally aligned 1, 2, 4, and 8 byte objects.

Atomic loads and stores of native lock-free widths use ordinary Bedrock load/store instructions for the memory access; the ABI tear-free guarantee for aligned 1, 2, 4, and 8 byte accesses is the source of atomicity. Relaxed load/store operations require no fence. Consume is treated as acquire. Acquire loads use a load followed by `AFENCE`; release stores use `AFENCE` followed by the store; sequentially consistent loads and stores use `AFENCE` on both sides until the ISA memory model defines a cheaper rule.

**Table 21. C Atomic Memory Order Mapping**

| C order | Bedrock order or sequence |
|---|---|
| relaxed | `relaxed` |
| consume | `acquire` |
| acquire | `acquire` |
| release | `release` |
| acq_rel | `acqrel` |
| seq_cst | `seqcst` |

Non-relaxed C fences lower to `AFENCE`; a relaxed thread fence is a no-op. Read-modify-write operations use the matching Bedrock atomic instruction for naturally aligned 1, 2, 4, and 8 byte objects.

**Table 23. C Atomic RMW Lowering**

| C operation | Lowering |
|---|---|
| `atomic_fetch_add` | `FETCHADD` |
| `atomic_fetch_sub` | `FETCHSUB` |
| `atomic_fetch_and` | `FETCHAND` |
| `atomic_fetch_or` | `FETCHOR` |
| `atomic_fetch_xor` | `FETCHXOR` |

| C operation | Lowering |
|---|---|
| `atomic_exchange` | compare-exchange loop |
| `atomic_compare_exchange` | `CMPXCHG` |

LLVM compare-exchange has separate success and failure orderings. The compiler selects the single Bedrock memory-order field that satisfies both after consume is mapped to acquire. Atomic helper names and exact helper signatures are owned by the compiler-runtime provider; once selected, helper calls use the normal Bedrock C ABI.

## 7.3 Volatile, Device, and Executable Memory

Volatile accesses are preserved as observable abstract-machine accesses, but volatile is not an ordering primitive and does not imply acquire, release, sequential consistency, cache synchronization, or device ordering. MMIO and device ordering are defined by the OS ABI, loader profile, or mapped memory attributes rather than by the C ABI.

Changes to executable memory after relocation, dynamic code patching, or JIT-style writes require explicit instruction/data cache synchronization before execution observes the modified instructions. A loader or runtime that modifies executable mappings is responsible for synchronization before transferring control to patched code.

# 8 Return Value Examples

## 8.1 C Binding Examples

The C binding uses a single return value class. Integer scalars return in D0, pointer and address scalars return in A0, and 128-bit integer scalars return in D1:D0 with the low 64 bits in D0. Aggregates up to 16 bytes return in D1:D0; larger aggregates use a caller-allocated return buffer.

**Integer scalar return**

```
uint64_t add64(uint64_t a, uint64_t b);
```

- D0 = return value

**Pointer return**

```
void *find_byte(void *base, uint64_t len, int byte);
```

- A0 = return pointer

**Floating-point scalar return**

```
double hypot2(double x, double y);
```

- F0 = return value

**128-bit scalar return**

```
unsigned __int128 multiply_wide(uint64_t a, uint64_t b);
```

- D0 = low 64 bits
- D1 = high 64 bits

**Small aggregate return**

```
struct pair make_pair(uint64_t first, uint64_t second);
```

- D0 = low-address 64 bits
- D1 = high-address 64 bits

**Large aggregate return**

```
struct triple make_triple(uint64_t a, uint64_t b, uint64_t c);
```

- caller allocates the return buffer and passes its pointer in A5
- callee stores the aggregate into the buffer named by A5 at entry
- callee returns the same buffer pointer in A0